МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
КРЕМЕНЧУЦЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ МИХАЙЛА ОСТРОГРАДСЬКОГО
НАВЧАЛЬНО-НАУКОВИЙ ІНСТИТУТ ЕЛЕКТРИЧНОЇ ІНЖЕНЕРІЇ
ТА ІНФОРМАЦІЙНИХ ТЕХНОЛОГІЙ

Кафедра автоматизації та інформаційних систем

ОСВІТНІЙ КОМПОНЕНТ
«ОРГАНІЗАЦІЯ БАЗ ДАНИХ»

ЗВІТ
З ЛАБОРАТОРНОЇ РОБОТИ № 3

Виконав:
студент групи КН-25-1
Рафієв Б. С.
Перевірив:
асистент кафедри АІС
Андреєв П.І.

Кременчук 2026

Тема: **Створення простих запитів, використання функцій.**

Мета: навчитися створювати прості запити до таблиць бази даних із використанням умов, впорядкування та групування результатів.

Порядок виконання роботи

Лабораторну роботу виконано в середовищі Microsoft SQL Server Management Studio 22. Для запитів першої частини використано навчальну базу даних Sales111, створену під час лабораторної роботи № 2; для індивідуального завдання — базу даних LibraryDB (варіант № 1 «Бібліотека»). Усі запити виконано через вікно New Query, результати наведено на відповідних рисунках.

1. Запити до навчальної бази даних Sales111

Нижче наведено запити, передбачені завданням, та результати їх виконання. Для наочності псевдоніми стовпців задано українською мовою.

1.1. Вивести прізвища всіх продавців, міста їхнього проживання та комісійні продавців, використовуючи замінник імен стовпців.

```
SELECT NAME_SP AS Прізвище,
       CITY_SP AS Місто,
       COMM_SP AS Комісійні
FROM Salespeople;
```

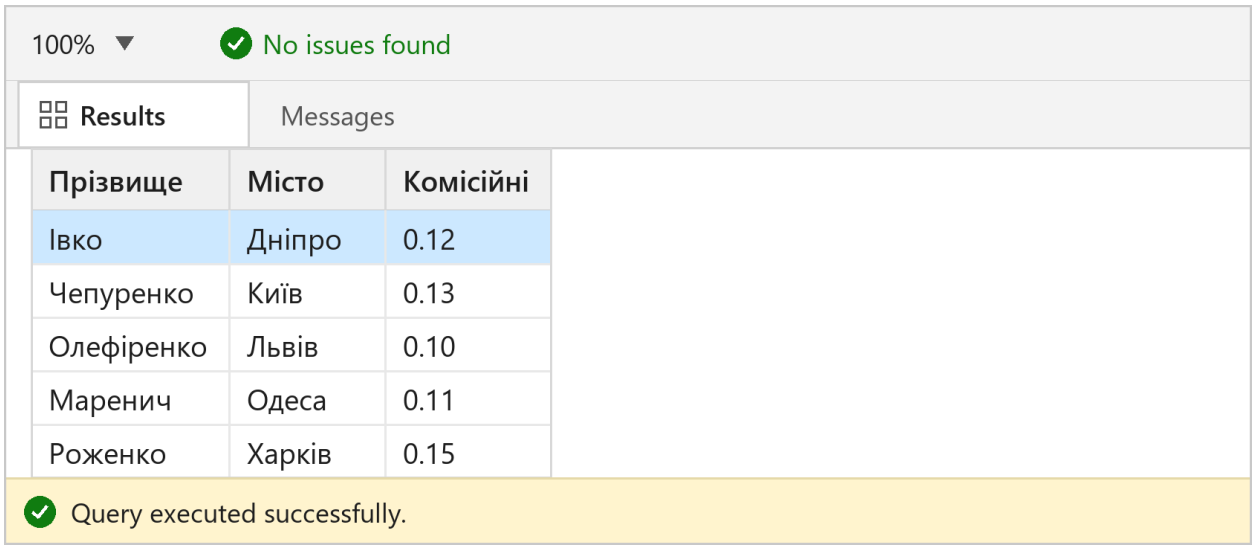


Рисунок 1 – Результат виконання запиту 1.1

Псевдоніми стовпців задано оператором AS.

1.2. Вивести коди усіх замовників та їхні прізвища за алфавітом.

```
SELECT ID_CM AS Код_замовника,
       NAME_CM AS Прізвище
FROM Customers
ORDER BY NAME_CM ASC;
```

100% No issues found

Results Messages

Код_замовника	Прізвище
2002	Гірченко
2004	Гора
2006	Клименко
2003	Луценко
2007	Петренко
2001	Хоменко
2008	Цапко
2005	Шевченко

Query executed successfully.

Рисунок 2 – Результат виконання запиту 1.2

Сортування за прізвищем виконано оператором ORDER BY ... ASC.

1.3. Вивести перелік міст, де мешкають замовники.

```
SELECT DISTINCT CITY_CM AS Місто
FROM Customers
ORDER BY CITY_CM;
```

100% No issues found

Results Messages

Місто
Бровари
Івано-Франківськ
Київ
Ромни
Чернігів

Query executed successfully.

Рисунок 3 – Результат виконання запиту 1.3

Оператор DISTINCT виключає повторення назв міст.

1.4. Вивести прізвища усіх продавців, що містять комбінації символів «ее» або «rr».

```
-- Основний запит: комбінації «ее» або «rr»
SELECT NAME_SP AS Прізвище
FROM Salespeople
WHERE NAME_SP LIKE '%ее%' OR NAME_SP LIKE '%rr%';

-- Адаптований запит: комбінації «ен» або «ов»
SELECT NAME_SP AS Прізвище
FROM Salespeople
WHERE NAME_SP LIKE '%ен%' OR NAME_SP LIKE '%ов%';
```

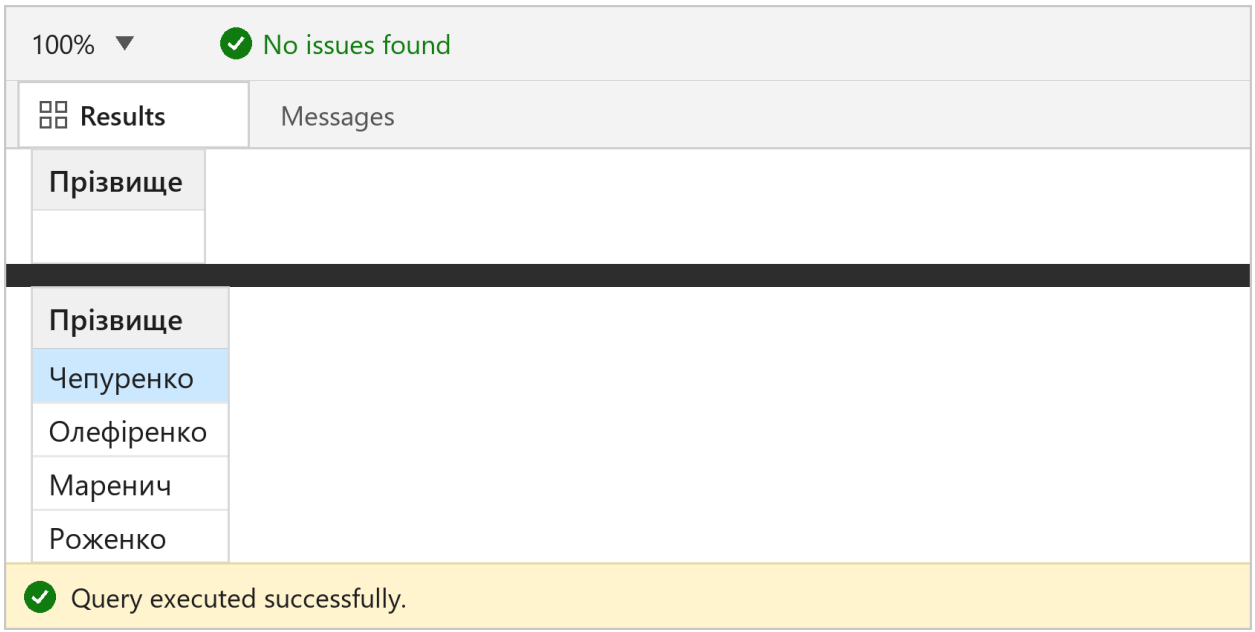


Рисунок 4 – Результати виконання запиту 1.4 (основний та адаптований запити)

У навчальній базі прізвища записані українською мовою, тому комбінації «ее» і «rr» не зустрічаються — основний запит повертає 0 рядків. Для перевірки роботи оператора LIKE наведено адаптований запит із сполученнями «ен» та «ов».

1.5. Вивести всю інформацію стосовно замовників, що мешкають у Лондоні.

```
-- Запит за умовою «Лондон»
SELECT ID_CM, NAME_CM, CITY_CM, BONUS_CM
FROM Customers
WHERE CITY_CM = 'Лондон';

-- Адаптований запит: замовники з Києва
SELECT ID_CM, NAME_CM, CITY_CM, BONUS_CM
FROM Customers
WHERE CITY_CM = 'Київ';
```

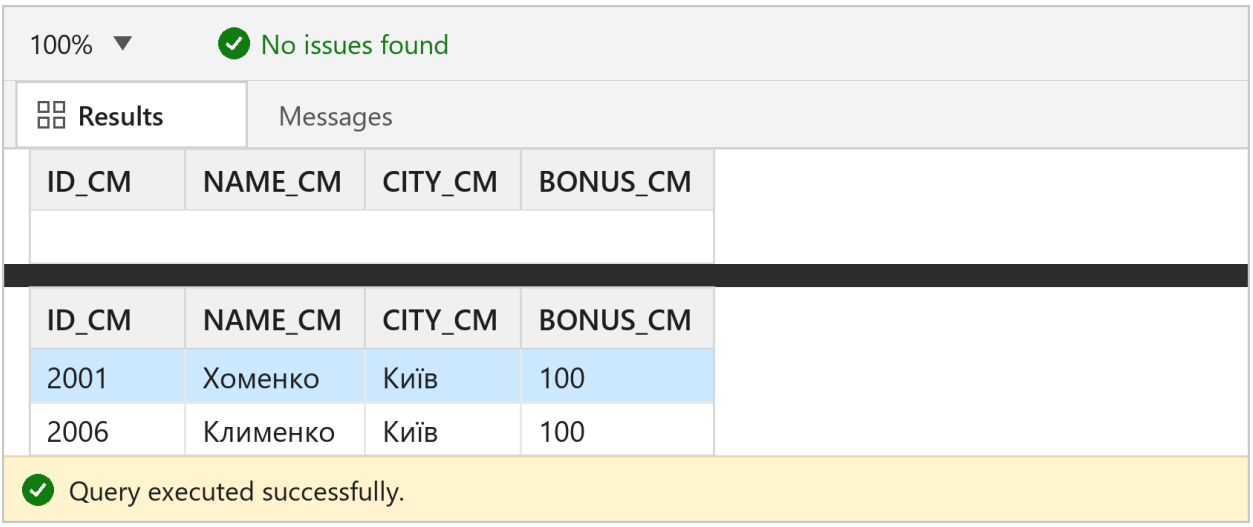


Рисунок 5 – Результати виконання запиту 1.5 (основний та адаптований запити)

У базі немає замовників із Лондона, тому основний запит повертає порожній результат. Додатково виконано аналогічний запит для міста Київ, яке присутнє в даних.

1.6. Вивести коди замовників, замовлення яких були зроблені у період з 10.04.2024 до 10.06.2024.

```
-- Заданий період 10.04.2024 – 10.06.2024
SELECT DISTINCT ID_CM AS Код_замовника
FROM Orders
WHERE DATE_OD BETWEEN '2024-04-10' AND '2024-06-10';

-- Адаптований період відповідно до наявних замовлень
SELECT DISTINCT ID_CM AS Код_замовника
FROM Orders
WHERE DATE_OD BETWEEN '2024-08-01' AND '2024-10-31';
```

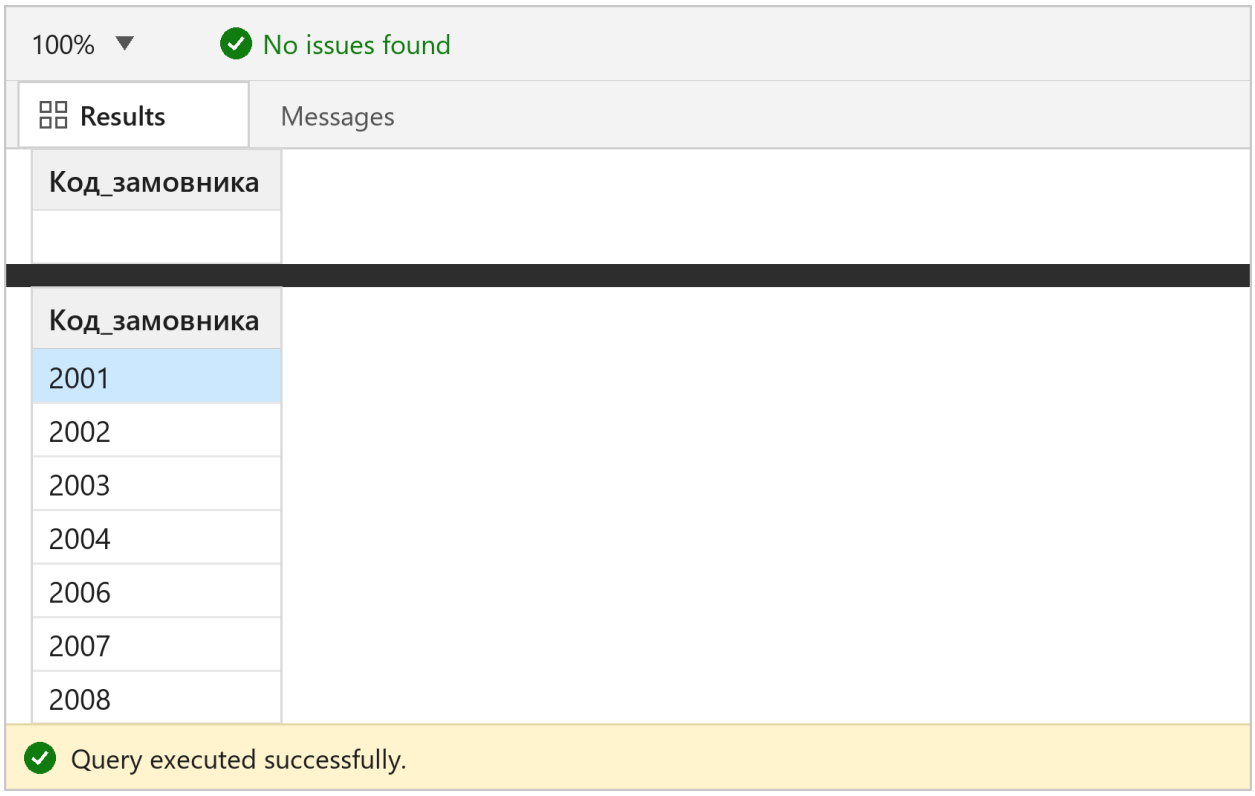


Рисунок 6 – Результати виконання запиту 1.6 (основний та адаптований запити)

Усі замовлення в базі датовані серпнем–жовтнем 2024 року, тому заданий у методичних вказівках період не містить записів. Для демонстрації відбору за датою використано період 01.08.2024–31.10.2024.

1.7. Вивести дані про замовників, які мають визначений рейтинг (Rating). Виконати запит кілька разів із різними рейтингами.

```
-- Замовники з рейтингом 100
SELECT ID_CM, NAME_CM, CITY_CM, BONUS_CM
FROM Customers
WHERE BONUS_CM = 100;

-- Замовники з рейтингом 300
SELECT ID_CM, NAME_CM, CITY_CM, BONUS_CM
FROM Customers
WHERE BONUS_CM = 300;
```

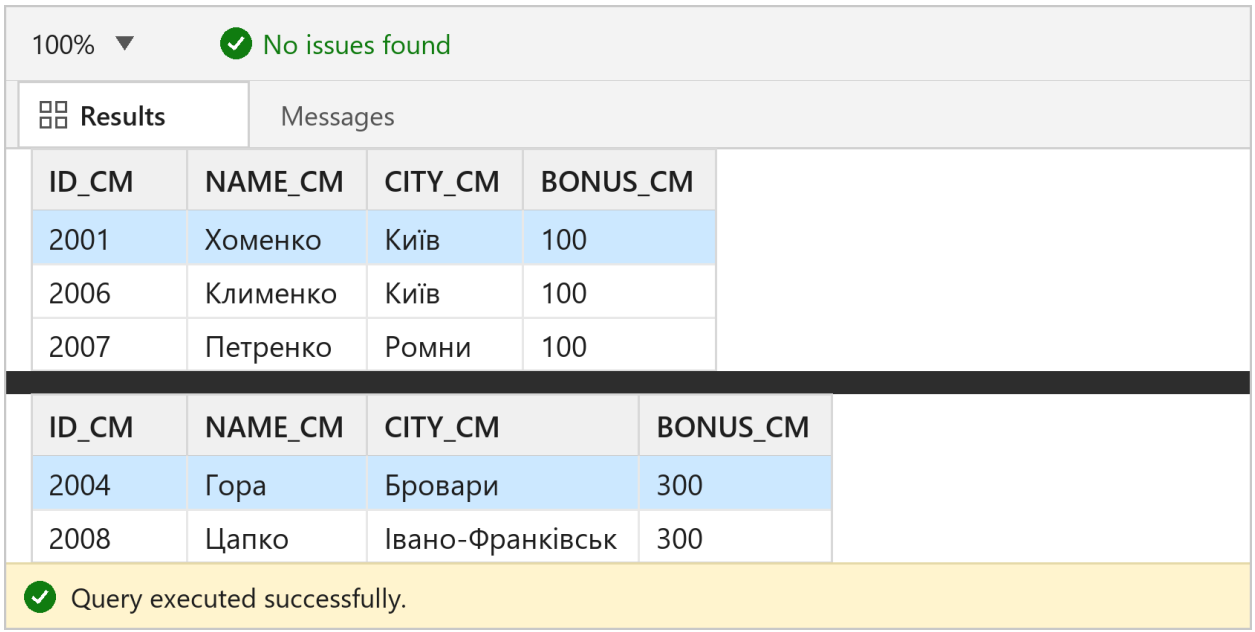
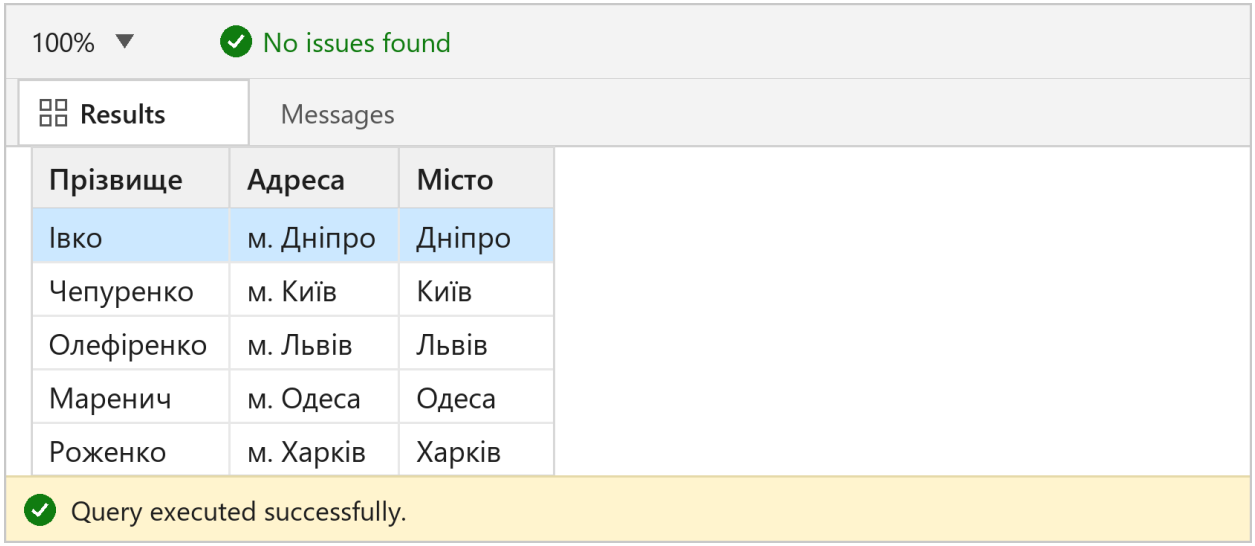


Рисунок 7 – Результати виконання запиту 1.7 з різними значеннями рейтингу

У створеній базі рейтинг зберігається у стовпці BONUS_CM. Запит виконано двічі: для рейтингу 100 та для рейтингу 300.

1.8. Вивести адреси і найменування міст із великої літери (таблиця продавців).

```
SELECT NAME_SP AS Прізвище,
       'м. ' + UPPER(LEFT(CITY_SP, 1)) +
       LOWER(SUBSTRING(CITY_SP, 2, LEN(CITY_SP))) AS Адреса,
       UPPER(LEFT(CITY_SP, 1)) +
       LOWER(SUBSTRING(CITY_SP, 2, LEN(CITY_SP))) AS Місто
FROM Salespeople;
```



Прізвище	Адреса	Місто
Івко	м. Дніпро	Дніпро
Чепуренко	м. Київ	Київ
Олефіренко	м. Львів	Львів
Маренич	м. Одеса	Одеса
Роженко	м. Харків	Харків

Рисунок 8 – Результат виконання запиту 1.8

У таблиці Salespeople немає окремого стовпця з адресою, тому адресу сформовано з назви міста. Першу літеру назви переведено у верхній регістр, решту — у нижній.

1.9. Скласти запити, що формують рядки: чотири перших символи «SQL_SERVER», починаючи з другого; вміст стовпця SName таблиці Salespeople, починаючи з другого символу.

```
SELECT SUBSTRING('SQL_SERVER', 2, 4) AS Результат;

SELECT SUBSTRING(NAME_SP, 2, LEN(NAME_SP)) AS Прізвище_з_2_символу
FROM Salespeople;
```

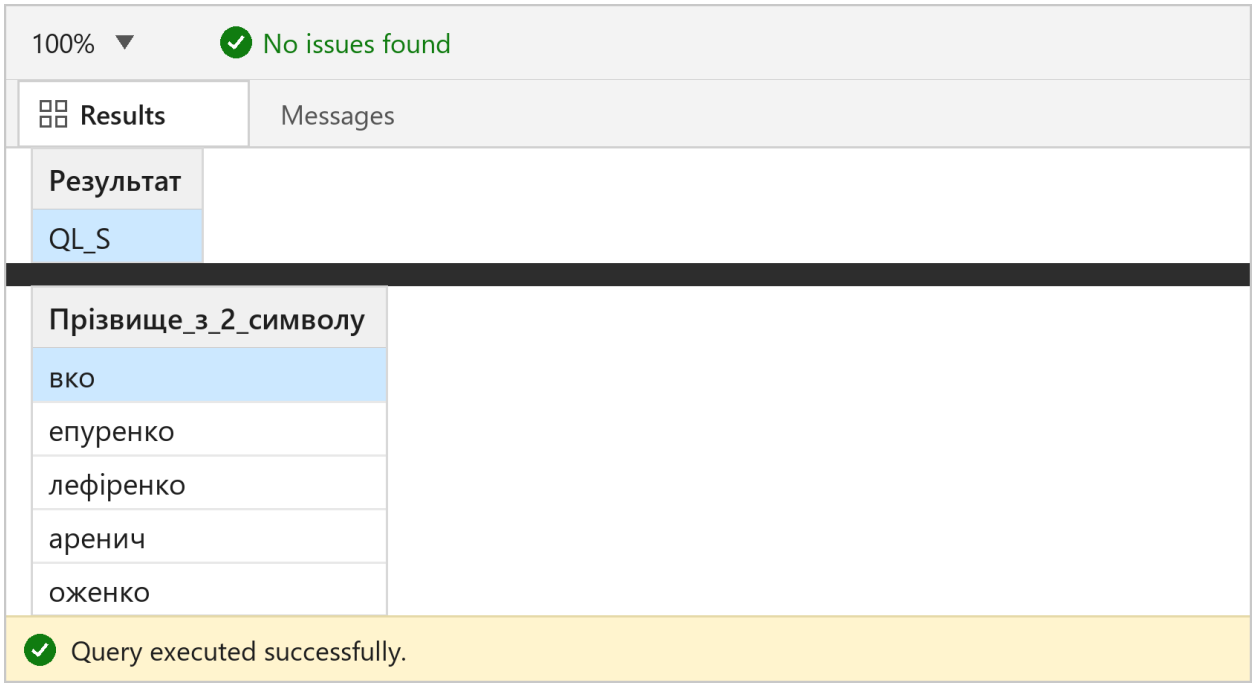


Рисунок 9 – Результати виконання запиту 1.9

Функція SUBSTRING повертає чотири символи рядка «SQL_SERVER», починаючи з другого: «QL_S». Для стовпця NAME_SP (у методичних вказівках — SName) виведено прізвище без першої літери.

1.10. Визначити, скільки прізвищ містять літеру «б» (таблиця замовників).

```
-- Кількість прізвищ із літерою «б»
SELECT COUNT(*) AS Кількість
FROM Customers
WHERE NAME_CM LIKE '%б%';

-- Адаптовано: кількість прізвищ із літерою «о»
SELECT COUNT(*) AS Кількість
FROM Customers
WHERE NAME_CM LIKE '%о%';
```

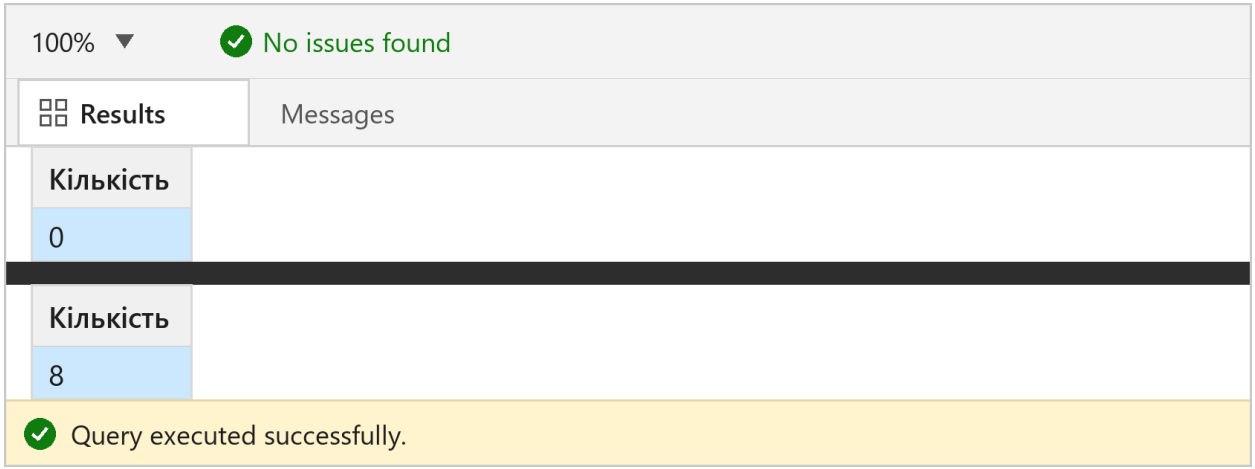


Рисунок 10 – Результати виконання запиту 1.10 (основний та адаптований запити)

У даних немає прізвищ із літерою «б», тому COUNT(*) дорівнює 0. Додатково підраховано кількість прізвищ із літерою «о».

1.11. Написати запит, що дозволяє одержати день тижня через два місяці після поточної дати.

```
SELECT DATEADD(MONTH, 2, GETDATE()) AS Дата_через_2_місяці,
       DATENAME(WEEKDAY, DATEADD(MONTH, 2, GETDATE())) AS День_тижня;
```

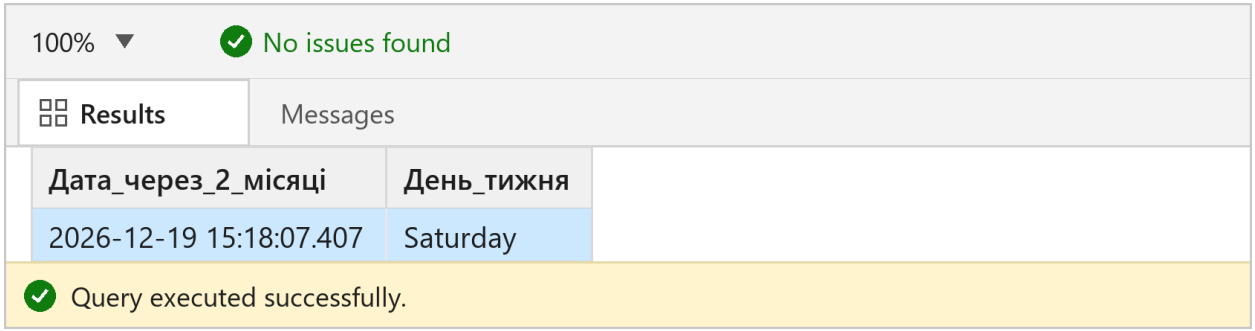


Рисунок 11 – Результат виконання запиту 1.11

Функція DATEADD додає два місяці до поточної дати, а DATENAME повертає назву дня тижня. Оскільки мовою сеансу MS SQL Server є англійська, назву дня виведено англійською (Saturday).

1.12. Вивести замовлення з мінімальною і максимальною вартістю та середню вартість замовлень.

```
SELECT MIN(AMOUNT_OD) AS Мінімальна_вартість,
       MAX(AMOUNT_OD) AS Максимальна_вартість,
       CAST(AVG(AMOUNT_OD) AS DECIMAL(10, 2)) AS Середня_вартість
FROM Orders;
```

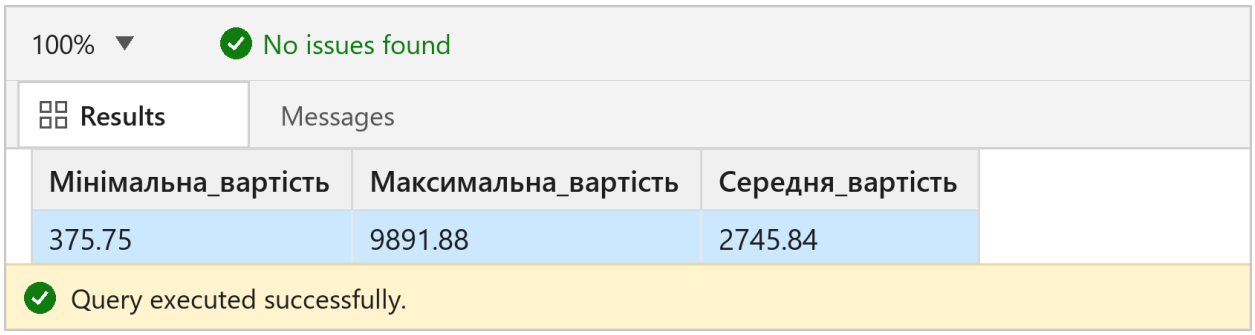


Рисунок 12 – Результат виконання запиту 1.12

Агрегатні функції MIN, MAX і AVG обчислено для стовпця AMOUNT_OD таблиці Orders.

1.13. Вивести кількість замовлень, що має кожний замовник.

```
SELECT ID_CM AS Код_замовника,
       COUNT(*) AS Кількість_замовлень
FROM Orders
GROUP BY ID_CM
ORDER BY Кількість_замовлень DESC, ID_CM;
```

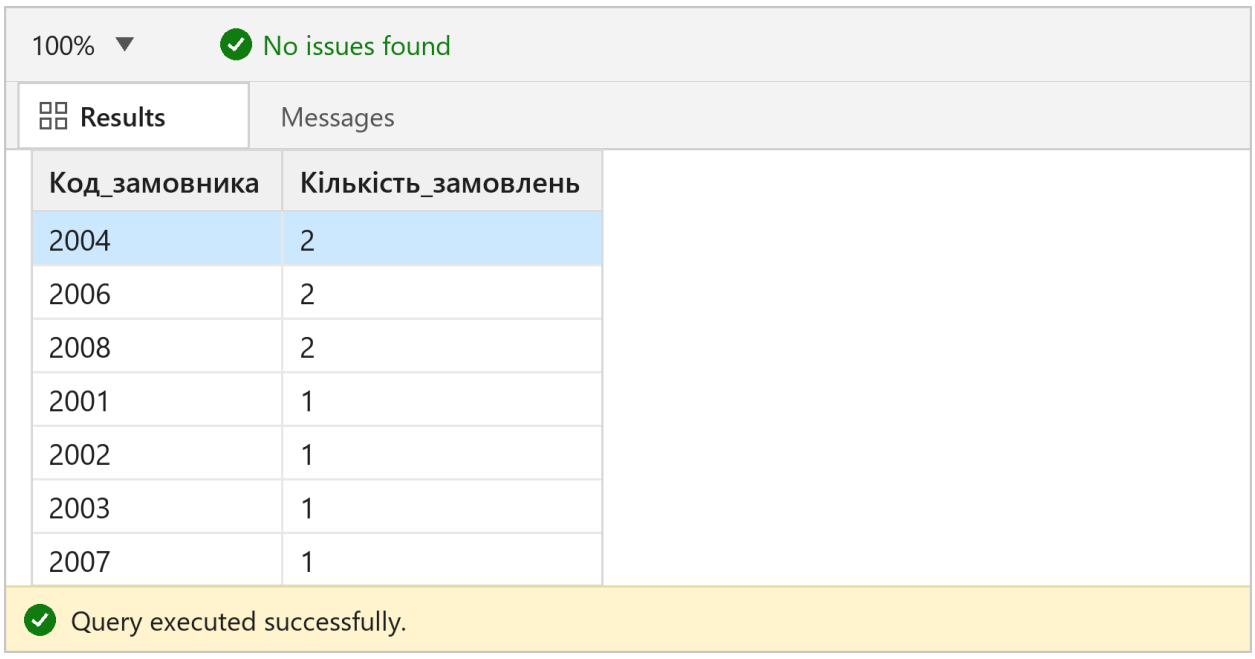


Рисунок 13 – Результат виконання запиту 1.13

Кількість замовлень згруповано за кодом замовника; результати відсортовано за спаданням кількості.

1.14. Вивести коди тільки тих замовників, які мають більше двох замовлень.

```
-- Замовники, які мають більше двох замовлень
SELECT ID_CM AS Код_замовника, COUNT(*) AS Кількість_замовлень
FROM Orders
GROUP BY ID_CM
HAVING COUNT(*) > 2;

-- Адаптовано: більше одного замовлення
SELECT ID_CM AS Код_замовника, COUNT(*) AS Кількість_замовлень
FROM Orders
GROUP BY ID_CM
HAVING COUNT(*) > 1;
```

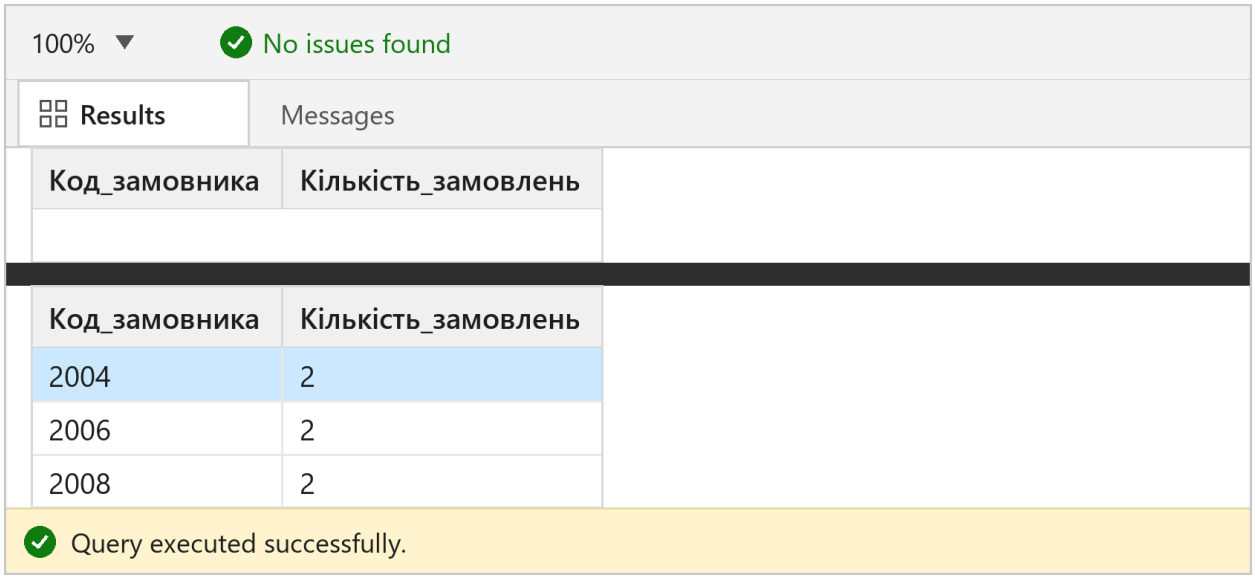


Рисунок 14 – Результати виконання запиту 1.14 (основний та адаптований запити)

Жоден замовник не має більше двох замовлень, тому основний запит повертає 0 рядків. Адаптований запит із умовою HAVING COUNT(*) > 1 виводить замовників із двома замовленнями.

2. Індивідуальне завдання. База даних LibraryDB

Індивідуальне завдання виконано для варіанта № 1 — предметна область «Бібліотека». Використано створену в лабораторній роботі № 2 базу даних LibraryDB, що містить таблиці Author, Publisher, Book, Reader і BookLoan. Для кожного пункту завдання складено по три різні запити.

2.1. Проста вибірка (вибір усіх значень з таблиці)

2.1.1. Вивести всі записи таблиці Author.

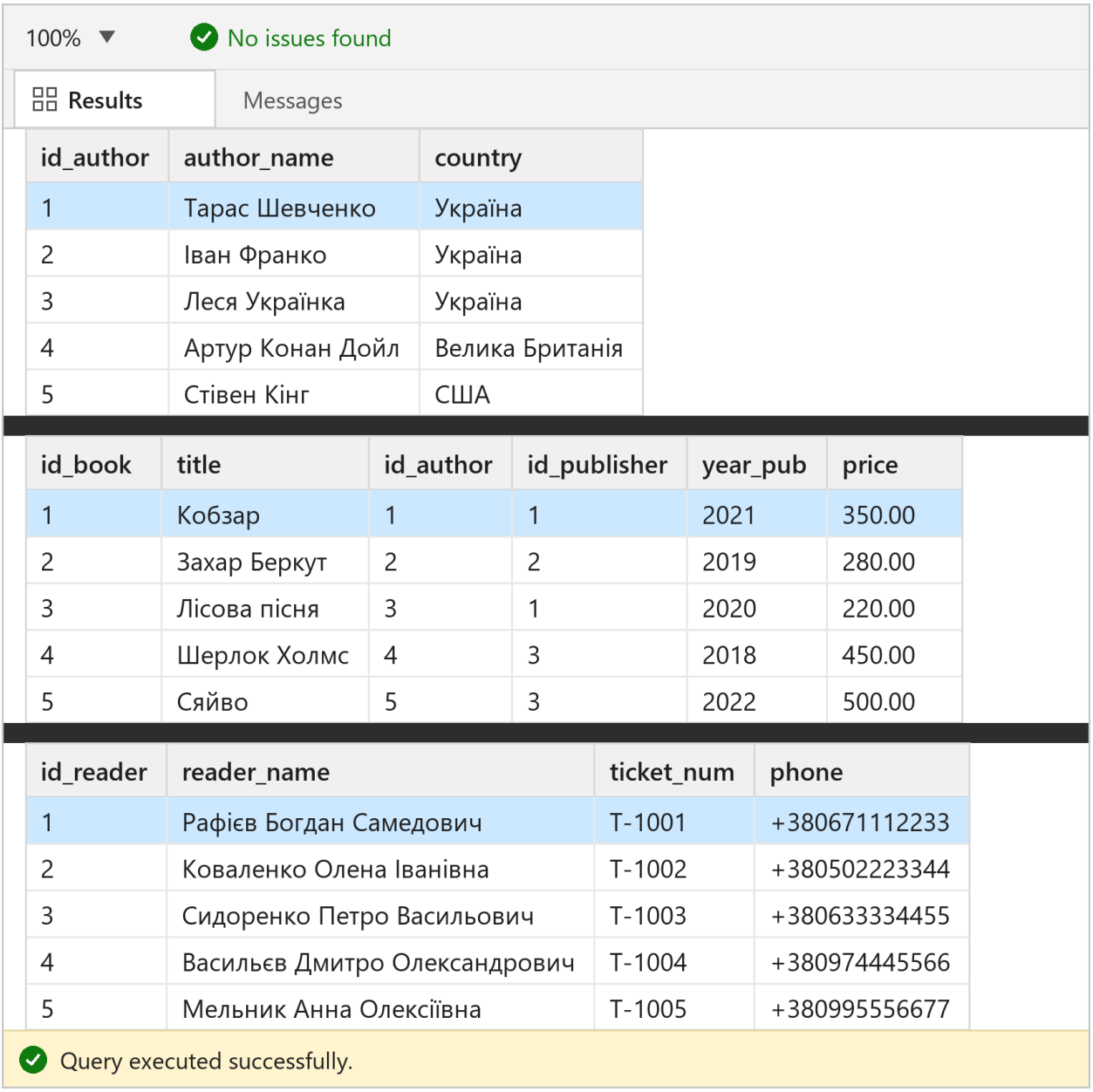
2.1.2. Вивести всі записи таблиці Book.

2.1.3. Вивести всі записи таблиці Reader.

```sql
-- 2.1.1. Усі автори
SELECT * FROM Author;

-- 2.1.2. Усі книги
SELECT * FROM Book;

-- 2.1.3. Усі читачі
SELECT * FROM Reader;
```



100% No issues found

Results Messages

id_author	author_name	country
1	Тарас Шевченко	Україна
2	Іван Франко	Україна
3	Леся Українка	Україна
4	Артур Конан Дойл	Велика Британія
5	Стівен Кінг	США

id_book	title	id_author	id_publisher	year_pub	price
1	Кобзар	1	1	2021	350.00
2	Захар Беркут	2	2	2019	280.00
3	Лісова пісня	3	1	2020	220.00
4	Шерлок Холмс	4	3	2018	450.00
5	Сяйво	5	3	2022	500.00

id_reader	reader_name	ticket_num	phone
1	Рафієв Богдан Самедович	T-1001	+380671112233
2	Коваленко Олена Іванівна	T-1002	+380502223344
3	Сидоренко Петро Васильович	T-1003	+380633334455
4	Васильєв Дмитро Олександрович	T-1004	+380974445566
5	Мельник Анна Олексіївна	T-1005	+380995556677

Query executed successfully.

Рисунок 15 – Результати виконання запитів 2.1.1–2.1.3

2.2. Вибірка обчислюваних значень (множення, додавання, віднімання, ділення)

2.2.1. Обчислити вартість двох примірників кожної книги.

2.2.2. Обчислити ціну книги зі знижкою 10 % та суму економії.

2.2.3. Обчислити вік кожної книги за роком видання.

```
-- 2.2.1. Вартість двох примірників
SELECT title AS Назва,
       price AS Ціна,
       CAST(price * 2 AS DECIMAL(10, 2)) AS Ціна_двох_примірників
FROM Book;

-- 2.2.2. Ціна зі знижкою 10 %
SELECT title AS Назва,
       price AS Ціна,
       CAST(price * 0.9 AS DECIMAL(10, 2)) AS Ціна_зі_знижкою,
       CAST(price * 0.1 AS DECIMAL(10, 2)) AS Економія
FROM Book;

-- 2.2.3. Вік книги
SELECT title AS Назва,
       year_pub AS Рік,
       YEAR(GETDATE()) - year_pub AS Вік_книги
FROM Book;
```

100% ▼ No issues found

Results | Messages

Назва	Ціна	Ціна_двох_примірників
Кобзар	350.00	700.00
Захар Беркут	280.00	560.00
Лісова пісня	220.00	440.00
Шерлок Холмс	450.00	900.00
Сяйво	500.00	1000.00

Назва	Ціна	Ціна_зі_знижкою	Економія
Кобзар	350.00	315.00	35.00
Захар Беркут	280.00	252.00	28.00
Лісова пісня	220.00	198.00	22.00
Шерлок Холмс	450.00	405.00	45.00
Сяйво	500.00	450.00	50.00

Назва	Рік	Вік_книги
Кобзар	2021	5
Захар Беркут	2019	7
Лісова пісня	2020	6
Шерлок Холмс	2018	8
Сяйво	2022	4

Query executed successfully.

Рисунок 16 – Результати виконання запитів 2.2.1–2.2.3

2.3. Вибір з умовою (WHERE)

2.3.1. Вивести книги, ціна яких перевищує 300 грн.

2.3.2. Вивести авторів із України.

2.3.3. Вивести книги, видані у 2020 році або пізніше.

```
-- 2.3.1. Книги дорожчі за 300 грн
SELECT title AS Назва, price AS Ціна
FROM Book
WHERE price > 300;

-- 2.3.2. Автори з України
SELECT author_name AS Автор, country AS Країна
FROM Author
WHERE country = 'Україна';

-- 2.3.3. Книги, видані з 2020 року
SELECT title AS Назва, year_pub AS Рік
FROM Book
WHERE year_pub >= 2020;
```

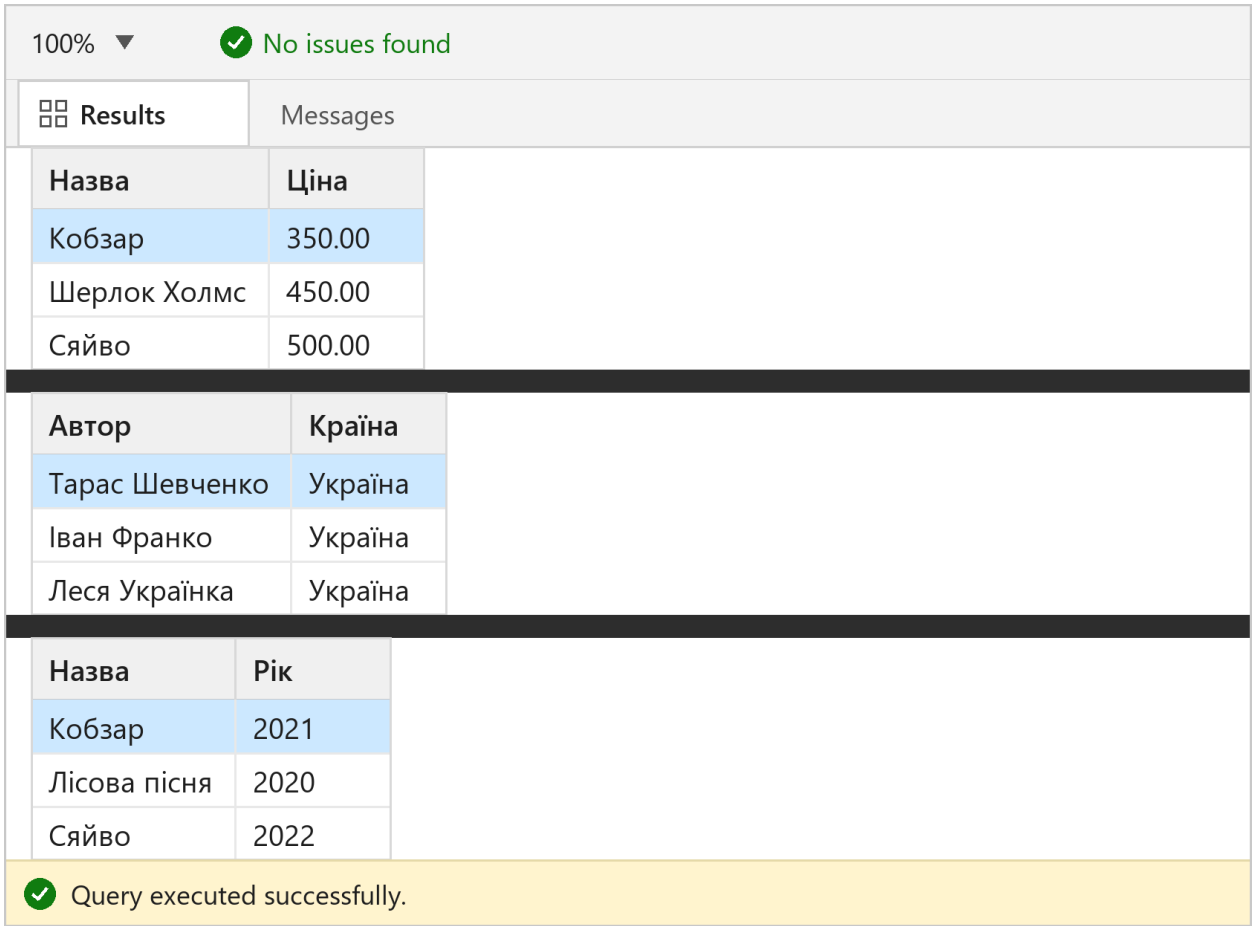


Рисунок 17 – Результати виконання запитів 2.3.1–2.3.3

2.4. Вибір з упорядкуванням (ORDER BY)

2.4.1. Вивести книги в порядку спадання ціни.

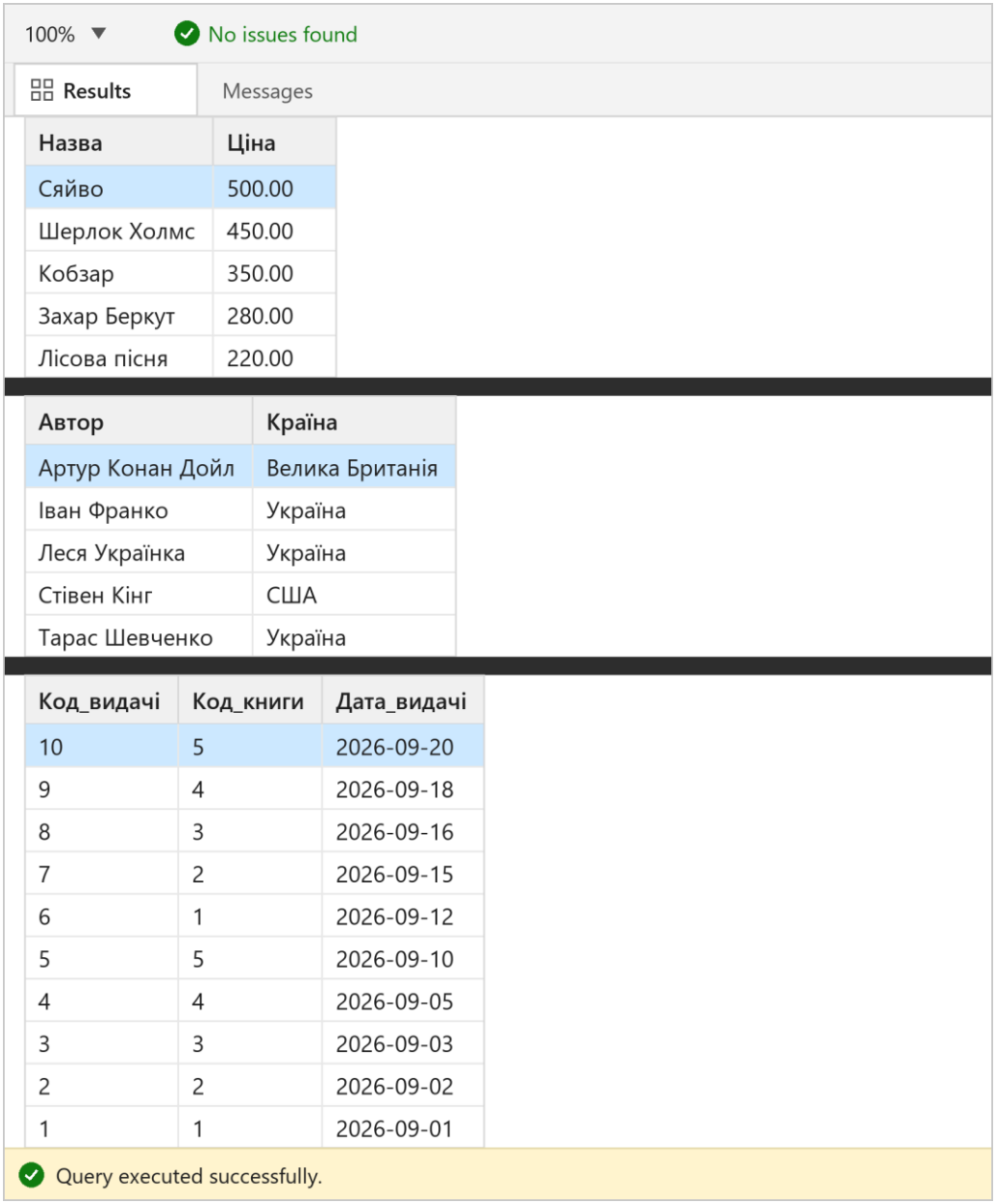
2.4.2. Вивести авторів за алфавітом.

2.4.3. Вивести видачі книг у порядку спадання дати видачі.

```
-- 2.4.1. Книги за спаданням ціни
SELECT title AS Назва, price AS Ціна
FROM Book
ORDER BY price DESC;

-- 2.4.2. Автори за алфавітом
SELECT author_name AS Автор, country AS Країна
FROM Author
ORDER BY author_name ASC;

-- 2.4.3. Видачі за спаданням дати
SELECT id_loan AS Код_видачі,
       id_book AS Код_книги,
       loan_date AS Дата_видачі
FROM BookLoan
ORDER BY loan_date DESC, id_book ASC;
```



Назва	Ціна
Сяйво	500.00
Шерлок Холмс	450.00
Кобзар	350.00
Захар Беркут	280.00
Лісова пісня	220.00

Автор	Країна
Артур Конан Дойл	Велика Британія
Іван Франко	Україна
Леся Українка	Україна
Стівен Кінг	США
Тарас Шевченко	Україна

Код_видачі	Код_книги	Дата_видачі
10	5	2026-09-20
9	4	2026-09-18
8	3	2026-09-16
7	2	2026-09-15
6	1	2026-09-12
5	5	2026-09-10
4	4	2026-09-05
3	3	2026-09-03
2	2	2026-09-02
1	1	2026-09-01

Рисунок 18 – Результати виконання запитів 2.4.1–2.4.3

2.5. Вибірка з використанням оператора BETWEEN

2.5.1. Вивести книги з ціною від 250 до 450 грн.

2.5.2. Вивести книги, видані у період з 2019 до 2021 року.

2.5.3. Вивести видачі за період з 05.09.2026 до 15.09.2026.

```
-- 2.5.1. Ціна в діапазоні 250–450 грн
SELECT title AS Назва, price AS Ціна
FROM Book
WHERE price BETWEEN 250 AND 450;

-- 2.5.2. Рік видання в діапазоні 2019–2021
SELECT title AS Назва, year_pub AS Рік
FROM Book
WHERE year_pub BETWEEN 2019 AND 2021;

-- 2.5.3. Дати видачі в заданому діапазоні
SELECT id_loan AS Код_видачі,
       loan_date AS Дата_видачі,
       return_date AS Дата_повернення
FROM BookLoan
WHERE loan_date BETWEEN '2026-09-05' AND '2026-09-15';
```

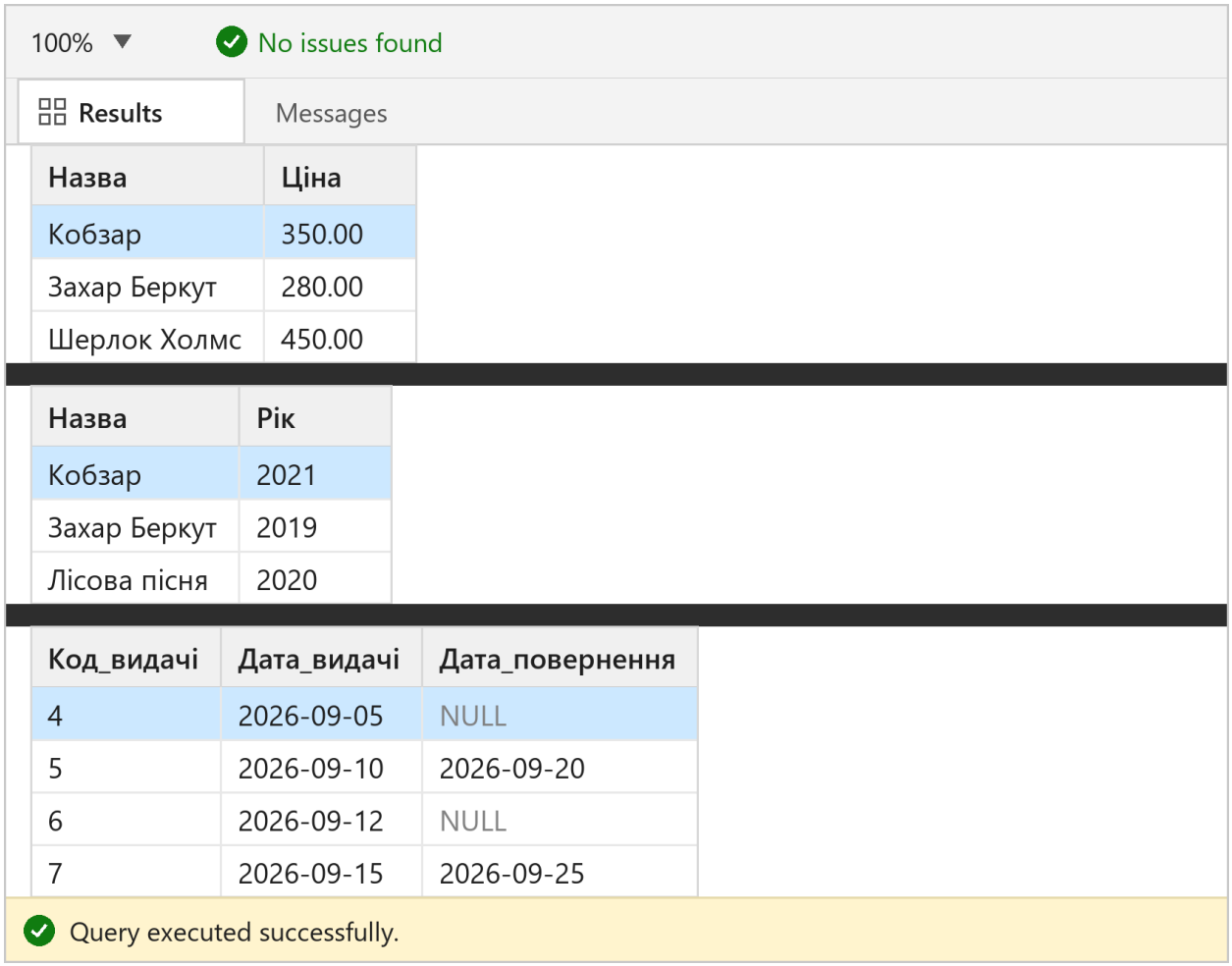


Рисунок 19 – Результати виконання запитів 2.5.1–2.5.3

2.6. Вибірка з використанням шаблона (LIKE)

2.6.1. Вивести авторів, ім’я яких починається на «Іван».

2.6.2. Вивести книги, у назві яких є літера «а».

2.6.3. Вивести читачів, номер телефону яких починається з «+38063».

```
-- 2.6.1. Автори, ім’я яких починається на «Іван»
SELECT author_name AS Автор
FROM Author
WHERE author_name LIKE 'Іван%';

-- 2.6.2. Книги, у назві яких є літера «а»
SELECT title AS Назва
FROM Book
WHERE title LIKE '%а%';

-- 2.6.3. Читачі з телефоном на «+38063»
SELECT reader_name AS Читач, phone AS Телефон
FROM Reader
WHERE phone LIKE '+38063%';
```

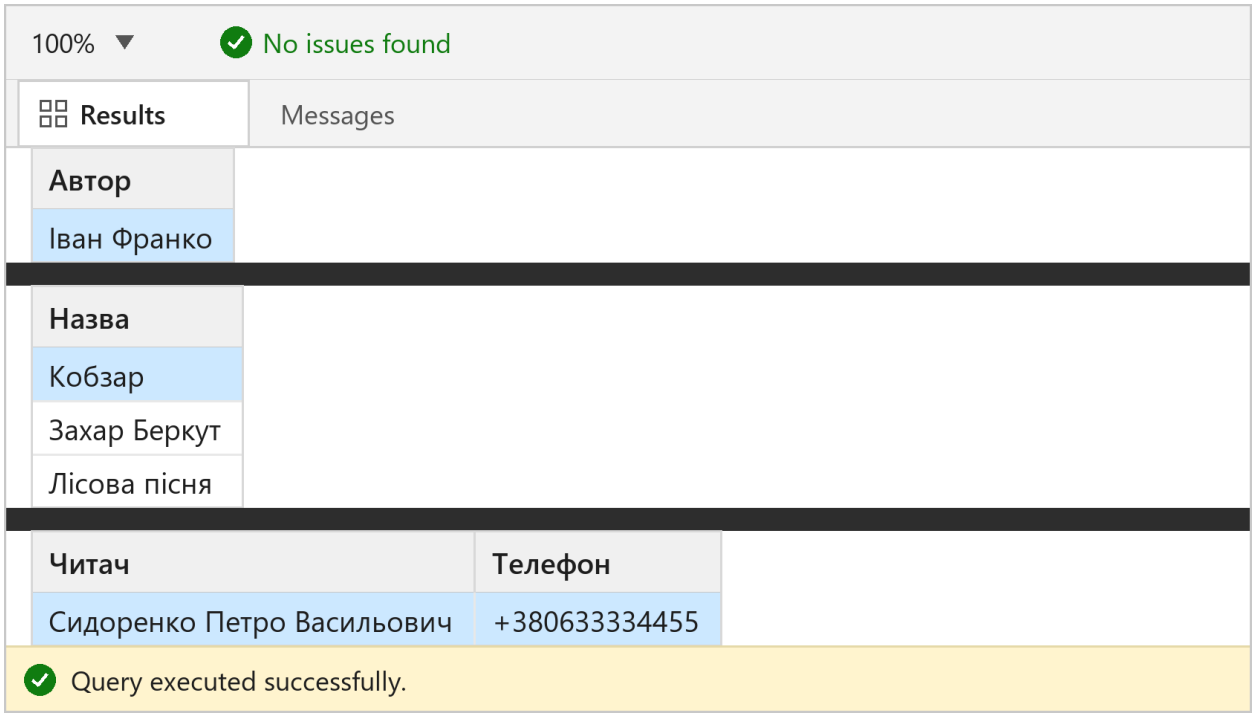


Рисунок 20 – Результати виконання запитів 2.6.1–2.6.3

2.7. Вибірка з використанням групування (GROUP BY)

2.7.1. Підрахувати кількість книг кожного видавництва.

2.7.2. Підрахувати кількість видач для кожного читача.

2.7.3. Підрахувати кількість книг за роками видання.

```
-- 2.7.1. Кількість книг за видавництвами
SELECT id_publisher AS Код_видавництва,
       COUNT(*) AS Кількість_книг
FROM Book
GROUP BY id_publisher
ORDER BY Кількість_книг DESC;

-- 2.7.2. Кількість видач за читачами
SELECT id_reader AS Код_читача,
       COUNT(*) AS Кількість_видач
FROM BookLoan
GROUP BY id_reader
ORDER BY Кількість_видач DESC;

-- 2.7.3. Кількість книг за роками
SELECT year_pub AS Рік,
       COUNT(*) AS Кількість_книг
FROM Book
GROUP BY year_pub
ORDER BY year_pub;
```

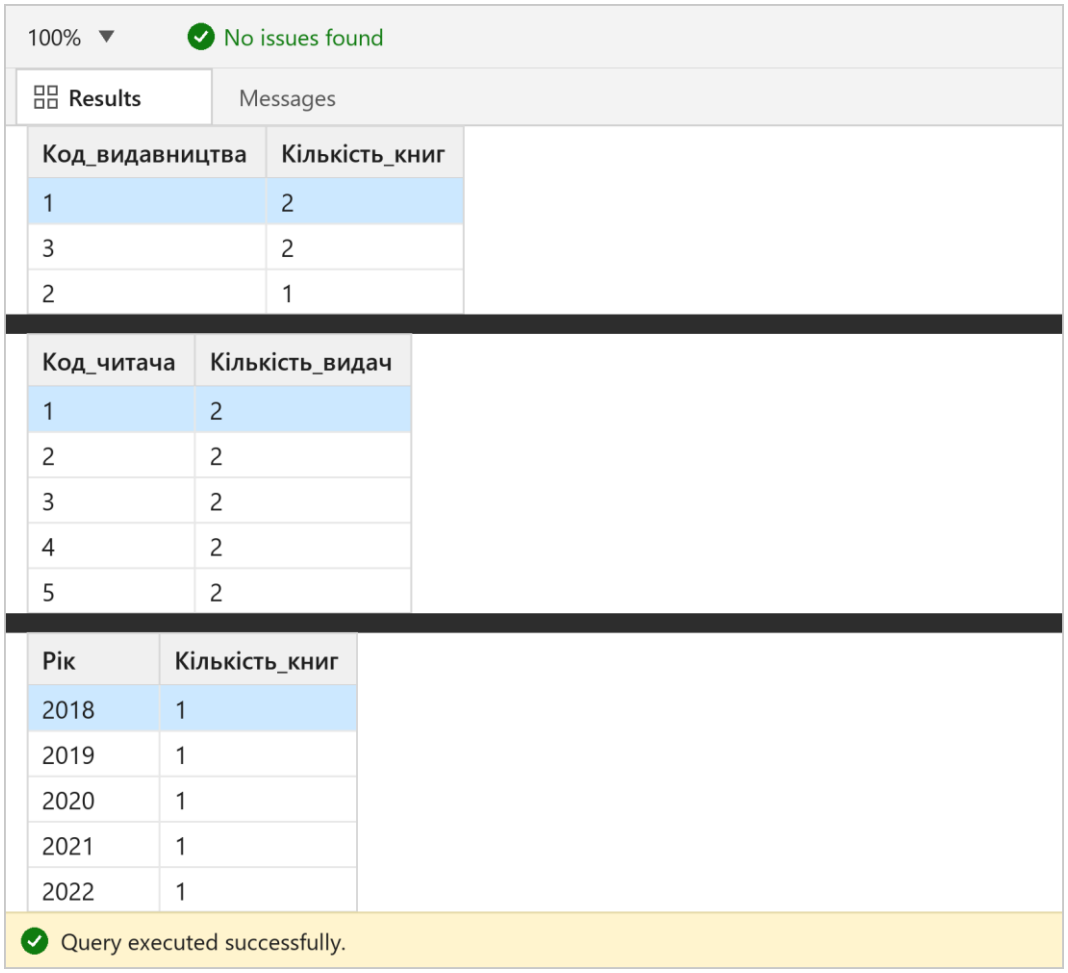


Рисунок 21 – Результати виконання запитів 2.7.1–2.7.3

2.8. Вибірка з використанням агрегатних функцій (COUNT, SUM, AVG)

2.8.1. Обчислити кількість книг, їхню загальну та середню вартість.

2.8.2. Обчислити мінімальну, максимальну та середню ціну книги.

2.8.3. Обчислити кількість і середню ціну книг, виданих з 2020 року.

```
-- 2.8.1. Кількість, сума та середня ціна книг
SELECT COUNT(*) AS Кількість_книг,
       SUM(price) AS Загальна_вартість,
       CAST(AVG(price) AS DECIMAL(10, 2)) AS Середня_ціна
FROM Book;

-- 2.8.2. Мінімальна, максимальна та середня ціна
SELECT MIN(price) AS Мінімальна_ціна,
       MAX(price) AS Максимальна_ціна,
       CAST(AVG(price) AS DECIMAL(10, 2)) AS Середня_ціна
FROM Book;

-- 2.8.3. Агрегати для книг, виданих з 2020 року
SELECT COUNT(*) AS Кількість_книг,
       CAST(AVG(price) AS DECIMAL(10, 2)) AS Середня_ціна
FROM Book
WHERE year_pub >= 2020;
```

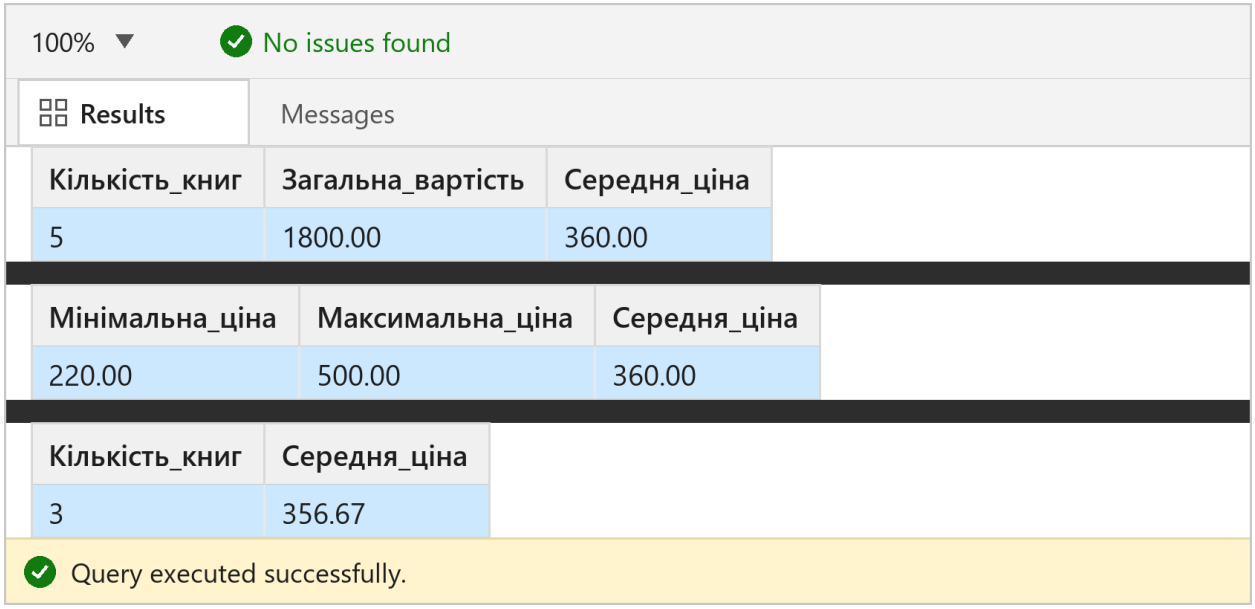


Рисунок 22 – Результати виконання запитів 2.8.1–2.8.3

Відповіді на контрольні питання

1. Наведіть синтаксис оператора SELECT.

```
SELECT список_стовпців_для_вибірки
    [ INTO нова_таблиця ]
    [ FROM джерело_таблиці ]
    [ WHERE умова_пошуку ]
    [ GROUP BY умова_групування ]
    [ HAVING умова_пошуку ]
    [ ORDER BY умова_сортування [ ASC | DESC ]]
```

Квадратними дужками позначено необов'язкові частини інструкції, а символом | — вибір одного з варіантів. Обов'язковими є лише оператори SELECT (які стовпці вибирати) та FROM (звідки вибирати дані); решта операторів включаються до запиту лише за необхідності. Після SELECT можна вказувати не лише стовпці таблиці, а й константи та вирази, що обчислюються під час виконання запиту. Приклад із виконаної роботи:

```
SELECT NAME_SP AS Прізвище, CITY_SP AS Місто
FROM Salespeople;
```

2. Як використовуються умови пошуку для відбору рядків?

Умова пошуку задається після оператора WHERE і являє собою логічний вираз, який обчислюється для кожного рядка джерела даних. До результату включаються тільки ті рядки, для яких цей вираз набуває значення TRUE; рядки, для яких умова хибна або невизначена (наприклад, через NULL), до вибірки не потрапляють. В умові використовуються операції порівняння (=, <>, >, <, >=, <=), логічні оператори AND, OR, NOT та спеціальні оператори IN, BETWEEN, LIKE і IS NULL. Кілька умов можна поєднувати в один вираз. Приклади з виконаної роботи:

```
SELECT title, price FROM Book WHERE price > 300;

SELECT ID_CM, NAME_CM, CITY_CM, BONUS_CM
FROM Customers
WHERE BONUS_CM = 100;
```

3. Які логічні оператори використовуються в умовах?

Основними логічними операторами є AND, OR і NOT. Оператор AND вимагає одночасної істинності обох умов, OR — істинності хоча б однієї з них, а NOT заперечує результат умови. За допомогою круглих дужок задають порядок

обчислення складених умов, оскільки AND має вищий пріоритет за OR. Крім цих операторів, у запитах використовуються операції порівняння (=, <>, >, <, >=, <=), а також спеціальні оператори IN, BETWEEN, LIKE та IS NULL, які фактично скорочують запис логічних виразів. Приклад:

```
-- прізвища продавців з комбінаціями «ее» або «rr»
SELECT NAME_SP FROM Salespeople
WHERE NAME_SP LIKE '%ee%' OR NAME_SP LIKE '%rr%';

-- книги дорожчі за 300 грн, видані після 2020 року
SELECT title, price FROM Book
WHERE price > 300 AND year_pub >= 2020;
```

4. Використання операторів IN, BETWEEN, LIKE.

Оператор IN визначає набір значень, яким має відповідати вираз, і замінює ланцюжок однотипних умов, з'єднаних оператором OR. Оператор BETWEEN задає діапазон за початковим і кінцевим значенням включно (межі входять до діапазону) і зручний для чисел, дат та рядків. Оператор LIKE відбирає рядки за шаблоном, у якому символ % відповідає будь-якій кількості будь-яких символів (у тому числі жодному), а символ _ — рівно одному символу. Перед будь-яким із цих операторів можна поставити NOT, щоб отримати протилежний результат. Приклади:

```
SELECT * FROM Customers
WHERE NAME_CM IN ('Макаренко', 'Іванина', 'Дмитренко');

SELECT * FROM Orders WHERE AMOUNT_OD BETWEEN 1000 AND 5000;

SELECT * FROM Salespeople WHERE NAME_SP LIKE 'Мак%';

SELECT title FROM Book WHERE title LIKE '%а%';
```

5. Як відсортувати результати запиту?

Сортування виконується оператором ORDER BY, після якого перелічують стовпці сортування через кому. За зростанням дані сортуються за замовчуванням (або явно за допомогою ASC), за спаданням — за допомогою DESC. Сортувати можна за декількома стовпцями: спочатку за першим, а в межах однакових значень — за наступними. У ORDER BY допускається використання псевдонімів стовпців зі списку SELECT та обчислюваних виразів. Якщо ORDER BY не вказано, порядок рядків у результатах вважається невизначеним. Приклад зі звіту:

```
SELECT ID_CM AS Код_замовника, COUNT(*) AS Кількість_замовлень
FROM Orders
GROUP BY ID_CM
ORDER BY Кількість_замовлень DESC, ID_CM ASC;
```

6. Як виконується групування результатів вибірки?

Групування виконується оператором GROUP BY і перетворює звичайний запит на підсумковий: рядки спочатку об'єднуються в групи за однаковими значеннями вказаних стовпців, а потім для кожної групи формується один підсумковий рядок. Якщо у списку SELECT разом зі стовпцями використовуються агрегатні функції, то перелік неагрегованих стовпців має бути обов'язково зазначений у GROUP BY. Групувати можна за декількома стовпцями одночасно. Оператор GROUP BY розташовується після WHERE, але перед HAVING та ORDER BY. Приклад із індивідуального завдання:

```
SELECT id_publisher AS Код_видавництва, COUNT(*) AS Кількість_книг
FROM Book
GROUP BY id_publisher;
```

7. Які агрегатні функції ви знаєте? Що обчислюють агрегатні функції?

Агрегатні функції обчислюють одне підсумкове значення для набору рядків вибірки. У MS SQL Server основними є: AVG — середнє значення числового стовпця; SUM — сума значень стовпця; MIN і MAX — відповідно найменше та найбільше значення; COUNT — кількість рядків. Функція COUNT(*) підраховує всі рядки вибірки, а COUNT(стовпець) — лише рядки, у яких значення цього стовпця не дорівнює NULL. Аргументом агрегатної функції може бути не лише ім'я стовпця, а й обчислюваний вираз, наприклад AVG(price * 2). Усі агрегатні функції, крім COUNT(*), ігнорують значення NULL. За замовчуванням обчислення виконуються для всіх рядків (ALL), проте оператор DISTINCT дозволяє враховувати лише унікальні значення. Агрегатні функції можна комбінувати в одному запиті:

```
SELECT COUNT(*) AS Кількість_книг,
       SUM(price) AS Загальна_вартість,
       MIN(price) AS Мінімальна_ціна,
       MAX(price) AS Максимальна_ціна,
       CAST(AVG(price) AS DECIMAL(10,2)) AS Середня_ціна
FROM Book;
```

8. Для чого використовується директива HAVING?

Оператор HAVING використовується разом з GROUP BY і визначає, які саме групи включаються до результатів запиту, тобто виконує фільтрацію груп подібно до того, як WHERE фільтрує окремі рядки. На відміну від WHERE, в умові HAVING можна використовувати агрегатні функції (COUNT, SUM, AVG, MIN, MAX), оскільки ця умова обчислюється вже після формування груп. Оператор HAVING записується після GROUP BY, а перед ORDER BY, якщо потрібне сортування. Приклад:

```
-- замовники, які мають більше двох замовлень
SELECT ID_CM AS Код_замовника, COUNT(*) AS Кількість_замовлень
FROM Orders
GROUP BY ID_CM
HAVING COUNT(*) > 2;
```

9. Які функції для роботи з датами ви знаєте?

Для роботи з датами та часом у MS SQL Server використовуються такі функції: GETDATE() — повертає поточні дату й час сервера; DATEADD(part, n, date) — додає до дати n одиниць заданої частини (DAY, MONTH, YEAR, HOUR тощо); DATEDIFF(part, date1, date2) — обчислює різницю між двома датами в заданих одиницях; DATENAME(part, date) і DATEPART(part, date) — повертають відповідно назву та числове значення частини дати (день тижня, місяць, рік); YEAR(date), MONTH(date), DAY(date) — виділяють відповідну частину дати. Крім того, для перетворення типів застосовуються CAST і CONVERT, а також функції DAY, ISDATE, EOMONTH і FORMAT. Функції дат часто застосовуються в умовах WHERE та в обчислюваних стовпцях:

```
SELECT DATEADD(MONTH, 2, GETDATE()) AS Дата_через_2_місяці,
       DATENAME(WEEKDAY, DATEADD(MONTH, 2, GETDATE())) AS День_тижня;

SELECT title, DATEDIFF(YEAR, year_pub, GETDATE()) AS Вік_книги
FROM Book;
```

10. Наведіть приклади використання функцій роботи з датами.

1. День тижня через два місяці після поточної дати (завдання 1.11):

```
SELECT DATEADD(MONTH, 2, GETDATE()) AS Дата_через_2_місяці,
       DATENAME(WEEKDAY, DATEADD(MONTH, 2, GETDATE())) AS День_тижня;
```

2. Коди замовників, замовлення яких зроблені у заданому періоді (завдання 1.6):

```
SELECT DISTINCT ID_CM AS Код_замовника
FROM Orders
WHERE DATE_OD BETWEEN '2024-04-10' AND '2024-06-10';
```

3. Вік кожної книги за роком видання (завдання 2.2.3):

```
SELECT title AS Назва,
       DATEDIFF(YEAR, year_pub, GETDATE()) AS Вік_книги
FROM Book;
```

4. Кількість і середня ціна книг, виданих починаючи з 2020 року (завдання 2.8.3):

```
SELECT COUNT(*) AS Кількість_книг,
       CAST(AVG(price) AS DECIMAL(10,2)) AS Середня_ціна
FROM Book
WHERE year_pub >= 2020;
```

У наведених прикладах DATEADD формує нову дату, DATEDIFF обчислює різницю у роках, DATENAME повертає назву дня тижня, а BETWEEN забезпечує відбір рядків за діапазоном дат.

Висновки:

Під час виконання лабораторної роботи № 3 було опановано створення простих запитів у MS SQL Server. Розглянуто оператори SELECT, WHERE, ORDER BY, GROUP BY, HAVING, оператори IN, BETWEEN, LIKE, IS NULL, агрегатні функції COUNT, SUM, AVG, MIN, MAX, а також рядкові та датні функції SUBSTRING, LEN, UPPER, LOWER, DATEADD і DATENAME. На прикладі баз Sales111 і LibraryDB виконано 14 запитів за методичними вказівками та 24 запити за індивідуальним варіантом. Отримані результати підтверджують коректність побудови запитів і дозволяють ефективно вибирати, фільтрувати, сортувати та агрегувати дані. Окремо надано відповіді на всі десять контрольних питань з методичних вказівок.